

ESCUELAS SALESIANAS MARÍA AUXILIADORA

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES

MULTIPLATAFORMA

PROYECTO FINAL - GRUPO 4: RUEDA FÁCIL

José Manuel Alonso Marín
SEVILLA,
2026

INDICE

1. Estudio del problema y análisis del sistema.
 - 1.1. Introducción
 - 1.2. Funciones y rendimientos deseados
 - 1.3. Objetivos
 - 1.4. Modelado de la solución
 - 1.4.1. Recursos humanos
 - 1.4.2. Recursos hardware
 - 1.4.3. Recursos software
2. Ejecución de la práctica
 - 2.1. Documentación técnica e implementación de la aplicación
3. Documentación del sistema
 - 3.1. Manual de instalación y configuración de la aplicación
 - 3.2. Manual de usuario
4. Conclusiones finales
 - 4.1. Grado de cumplimiento de los objetivos fijados
 - 4.2. Propuesta de modificaciones o ampliaciones futuras del sistema implementado
5. Bibliografía

1. ESTUDIO DEL PROBLEMA Y ANÁLISIS DEL SISTEMA

1.1. INTRODUCCIÓN

El proyecto se basa en la empresa Ruedafácil. Se dedican al alquiler de vehículos para particulares y empresas, y recientemente han decidido informatizar la gestión de su actividad diaria. La información proporcionada es la siguiente:

- Cada cliente debe estar identificado por su nombre completo, DNI, teléfono, correo electrónico, dirección y número de carnet de conducir. Los clientes podrán alquilar distintos vehículos, por lo que será importante conservar un historial de sus alquileres.
- Los vehículos disponibles deberán registrarse con su matrícula única, marca, modelo, año de fabricación, tipo de combustible, número de plazas, precio por día y estado actual, por ejemplo disponible, alquilado, en mantenimiento o dado de baja. Esta información permitirá controlar correctamente la flota de la empresa.
- El personal de la empresa está compuesto por distintos empleados, de los cuales se necesita conocer su nombre completo, cargo, oficina en la que trabaja, turno y años de experiencia.
- Cada alquiler realizado quedará registrado en el sistema, incluyendo la fecha de inicio, fecha de devolución prevista, fecha de devolución real, cliente que alquila el vehículo, empleado responsable, vehículo alquilado, precio total y estado del contrato.
- Cada vehículo se clasificará según su naturaleza: turismo, furgoneta, motocicleta, vehículo eléctrico, todoterreno, vehículo de lujo, etc. Cada vehículo registrado deberá estar asociado obligatoriamente a una categoría específica.

1.2. FUNCIONES Y RENDIMIENTOS DESEADOS

El programa debe permitir la gestión y funcionalidad de los clientes, empleados, vehículos, sus categorías y alquileres:

- Clientes: Crear, leer, actualizar y borrar clientes de la base de datos.
- Empleados: Crear, leer, actualizar y borrar empleados de la base de datos.
- Vehículos: Crear, leer, actualizar y borrar vehículos de la base de datos. Adicionalmente, se debe poder consultar vehículos disponibles por categoría.
- Categorías: Crear, leer, actualizar y borrar categorías de la base de datos.
- Alquileres: Crear, leer, actualizar y borrar alquileres de la base de datos. Adicionalmente, se debe poder consultar el historial de alquileres de un cliente concreto.

1.3. OBJETIVOS

El objetivo es desarrollar un programa que permita gestionar totalmente toda la información de clientes, empleados, alquileres, vehículos y sus categorías. El programa debe facilitar el acceso y modificación de la información de la base de datos.

1.4. MODELADO DE LA SOLUCIÓN

1.4.1 RECURSOS HUMANOS

No hay recursos humanos como inicios de sesión, etc...

1.4.2 RECURSOS HARDWARE

Los recursos necesarios son un ordenador o portátil y un servidor para la base de datos, ya sea remoto o local

1.4.2 RECURSOS SOFTWARE

- JRE (Java Runtime Environment): Entorno para ejecutar el programa.
- MySQL: Gestor de bases de datos utilizado para almacenar la información de las diferentes entidades.
- MySQL Connector: Permite la comunicación entre el programa y la base de datos.

2. EJECUCIÓN DE LA PRÁCTICA

2.1 DOCUMENTACIÓN TÉCNICA E IMPLEMENTACIÓN DE LA APLICACIÓN

Para la implementación de la aplicación hay que distinguir entre el patrón de diseño para el programa Java y el de la base de datos:

BASE DE DATOS:

El gestor utilizado es MySQL, ya que cumple con el requisito de emplear una base de datos relacional SQL. La base de datos está implementada en un servidor local.

Para el diseño de la base de datos se ha creado el siguiente Diagrama E-R y Modelo Relacional, con las entidades y relaciones representadas:

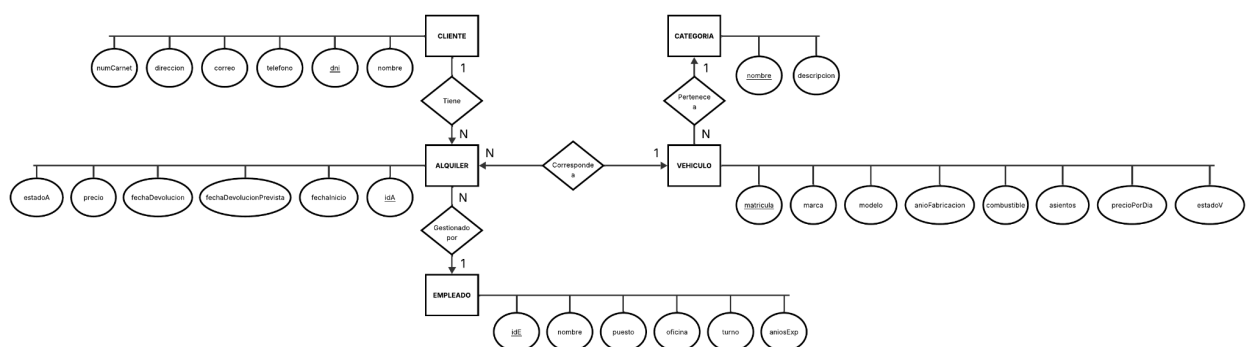


Diagrama E-R

El diagrama está compuesto por las entidades CLIENTE, EMPLEADO, VEHICULO, ALQUILER y CATEGORIA. Todas las relaciones entre entidades son 1:n.

Modelo relacional:

CLIENTE(dni, nombre, telefono, correo, direccion, numCarnet)

PK: DNI_alumno

EMPEADO(idE, nombre, puesto, oficina, turno, aniosExp)

PK: DNI_profesor

VEHICULO(matricula, marca, modelo, anioFabricacion, combustible, asientos, precioPorDia, estadoV, nombre)

PK: matricula

FK: nombre REFERENCES CATEGORIA (nombre)

CATEGORIA(nombre, descripcion)

PK: nombre

ALQUILER(idA, fechaInicio, fechaDevolucionPrevista, fechaDevolucion, precio, estadoA, dni, idE, matricula)

PK: idA

FK: dni REFERENCES CLIENTE (dni)

FK: idE REFERENCES EMPLEADO (idE)

FK: matricula REFERENCES VEHICULO (matricula)

Modelo Relacional

El script SQL con el que se ha creado la base de datos es el siguiente:

Creación de la base de datos

```
CREATE DATABASE RUEDAFACIL;
```

Creación de tablas

```
CREATE TABLE CATEGORIA (  
    nombre VARCHAR(50) PRIMARY KEY,  
    descripcion TEXT  
);
```

```
CREATE TABLE CLIENTE (  
    dni VARCHAR(15) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    telefono VARCHAR(20),  
    correo VARCHAR(100),  
    direccion VARCHAR(200),  
    numCarnet VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE EMPLEADO (  
    idE INT AUTO_INCREMENT PRIMARY KEY,
```

```
nombre VARCHAR(100) NOT NULL,  
puesto VARCHAR(50),  
oficina VARCHAR(50),  
turno VARCHAR(50),  
aniosExp INT  
);
```

```
CREATE TABLE VEHICULO (  
matricula VARCHAR(15) PRIMARY KEY,  
marca VARCHAR(50) NOT NULL,  
modelo VARCHAR(50) NOT NULL,  
anioFabricacion INT,  
combustible VARCHAR(50),  
asientos INT,  
precioPorDia DECIMAL(10,2) NOT NULL,  
estadoV VARCHAR(50),  
categoria VARCHAR(50),  
FOREIGN KEY (categoria) REFERENCES CATEGORIA(nombre) ON UPDATE CASCADE  
ON DELETE SET NULL  
);
```

```
CREATE TABLE ALQUILER (  
idA INT AUTO_INCREMENT PRIMARY KEY,  
fechaInicio DATE NOT NULL,  
fechaDevolucionPrevista DATE NOT NULL,  
fechaDevolucion DATE,  
precio DECIMAL(10,2),  
estadoA VARCHAR(50),  
dni VARCHAR(15),  
idE INT,  
matricula VARCHAR(15),  
FOREIGN KEY (dni) REFERENCES CLIENTE(dni) ON UPDATE CASCADE ON DELETE  
CASCADE,  
FOREIGN KEY (idE) REFERENCES EMPLEADO(idE) ON UPDATE CASCADE ON DELETE  
SET NULL,  
FOREIGN KEY (matricula) REFERENCES VEHICULO(matricula) ON UPDATE CASCADE  
ON DELETE CASCADE  
);
```

Algunos datos insertados en las tablas

```
INSERT INTO CATEGORIA (nombre, descripcion) VALUES  
( 'Turismo', 'Vehículos estándar y compactos, ideales para el uso diario y urbano.' ),  
( 'Furgoneta', 'Vehículos comerciales de gran capacidad, perfectos para mudanzas, carga o  
grupos grandes.' ),  
( 'Motocicleta', 'Vehículos de dos ruedas, muy ágiles y económicos para desplazarse  
rápidamente por la ciudad.' ),
```

('Vehículo eléctrico', 'Coches 100% ecológicos con cero emisiones, tecnología avanzada y etiqueta ambiental Cero.'),
('Todoterreno', 'Vehículos potentes con tracción 4x4, diseñados para circular por terrenos difíciles, arena o campo.'),
('De lujo', 'Modelos de alta gama, deportivos o berlinas de representación con las máximas prestaciones y confort.');

```
INSERT INTO CLIENTE (dni, nombre, telefono, correo, direccion, numCarnet) VALUES  
( '12345678A', 'Juan Pérez Gómez', '600112233', 'juan.perez@email.com', 'Calle Mayor 15, Sevilla', 'CARNET-ABC12'),  
( '87654321B', 'María López Torres', '655443322', 'maria.lopez@email.com', 'Av. Constitución 42, Dos Hermanas', 'CARNET-XYZ34'),  
( '45678912C', 'Carlos Ruiz Marín', '611998877', 'carlos.ruiz@email.com', 'Calle Betis 8, Triana', 'CARNET-LMN56');
```

```
INSERT INTO EMPLEADO (nombre, puesto, oficina, turno, aniosExp) VALUES  
( 'Ana Martínez Lara', 'Recepcionista', 'Oficina Central', 'Maniana', 4),  
( 'Luis Gómez Sancho', 'Gestor de Flota', 'Oficina Aeropuerto', 'Tarde', 7),  
( 'Elena Ortiz Vaca', 'Asesora Comercial', 'Oficina Estación', 'Noche', 2);
```

```
INSERT INTO VEHICULO (matricula, marca, modelo, anioFabricacion, combustible, asientos, precioPorDia, estadoV, categoria) VALUES  
( '1111AAA', 'Seat', 'Ibiza', 2022, 'Gasolina', 5, 35.00, 'Disponible', 'Turismo'),  
( '2222BBB', 'Renault', 'Traffic', 2021, 'Diesel', 9, 70.00, 'Disponible', 'Furgoneta'),  
( '3333CCC', 'Tesla', 'Model Y', 2024, 'Electrico', 5, 95.00, 'Alquilado', 'Vehículo eléctrico');
```

```
INSERT INTO ALQUILER (fechaInicio, fechaDevolucionPrevista, fechaDevolucion, precio, estadoA, dni, idE, matricula) VALUES  
( '2026-05-01', '2026-05-05', '2026-05-05', 140.00, 'Finalizado', '12345678A', 1, '1111AAA'),  
( '2026-05-20', '2026-05-25', NULL, 475.00, 'Activo', '87654321B', 2, '3333CCC'),  
( '2026-06-01', '2026-06-03', NULL, 140.00, 'Activo', '45678912C', 3, '2222BBB');
```

Control de la base de datos

Para controlar la base de datos, he creado 4 funciones y 4 procedimientos.

PROCEDIMIENTOS:

```
-- PROCEDIMIENTO finalizar_alquiler  
-- Registra la fecha de devolución y calcula el precio total. Usa JOIN.  
DELIMITER $$  
CREATE PROCEDURE finalizar_alquiler(IN p_idA INT, IN p_fechaDev DATE)  
BEGIN  
    DECLARE v_dias INT;  
    DECLARE v_precioDiario DECIMAL(10,2);  
    DECLARE v_matricula VARCHAR(15);  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN SELECT 'Error en la  
actualización del alquiler' AS mensaje; END;
```

```
SELECT a.matricula, v.precioPorDia INTO v_matricula, v_precioDiario
FROM ALQUILER a JOIN VEHICULO v ON a.matricula = v.matricula
WHERE a.idA = p_idA;

SET v_dias = DATEDIFF(p_fechaDev, (SELECT fechaInicio FROM ALQUILER WHERE idA
= p_idA));
IF v_dias < 1 THEN SET v_dias = 1; END IF;

UPDATE ALQUILER
SET fechaDevolucion = p_fechaDev,
    precio = (v_dias * v_precioDiario),
    estadoA = 'Completado'
WHERE idA = p_idA;

UPDATE VEHICULO SET estadoV = 'Disponible' WHERE matricula = v_matricula;
END $$
DELIMITER ;

-- PROCEDIMIENTO listar_vehiculos_categoria
-- Usa un cursor y JOIN para listar vehículos de una categoría.
DELIMITER $$
CREATE PROCEDURE listar_vehiculos_categoria(IN p_cat_nombre VARCHAR(50))
BEGIN
    DECLARE v_mat VARCHAR(15);
    DECLARE v_modelo VARCHAR(50);
    DECLARE fin_cursor INT DEFAULT 0;
    DECLARE cur_v CURSOR FOR
        SELECT v.matricula, v.modelo
        FROM VEHICULO v JOIN CATEGORIA c ON v.categoria = c.nombre
        WHERE c.nombre = p_cat_nombre;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = 1;

    OPEN cur_v;
bucle: LOOP
    FETCH cur_v INTO v_mat, v_modelo;
    IF fin_cursor = 1 THEN LEAVE bucle; END IF;
    SELECT CONCAT('Vehículo: ', v_mat, ' - Modelo: ', v_modelo) AS info;
END LOOP;
CLOSE cur_v;
END $$
DELIMITER ;

-- PROCEDIMIENTO asignar_empleado_alquiler
-- Cambia el empleado responsable de un alquiler existente.
DELIMITER $$
CREATE PROCEDURE asignar_empleado_alquiler(IN p_idA INT, IN p_idE INT)
BEGIN
```



```
DECLARE v_existe INT;  
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN SELECT 'Error al reasignar  
empleado' AS mensaje; END;
```

```
SELECT COUNT(*) INTO v_existe FROM EMPLEADO WHERE idE = p_idE;  
IF v_existe = 0 THEN  
    SELECT 'El empleado no existe' AS mensaje;  
ELSE  
    UPDATE ALQUILER SET idE = p_idE WHERE idA = p_idA;  
    SELECT 'Empleado reasignado correctamente' AS mensaje;  
END IF;  
END $$  
DELIMITER ;
```

```
-- PROCEDIMIENTO aplicar_descuento_categoria  
-- Incrementa el precio por día de todos los vehículos de una categoría.  
DELIMITER $$  
CREATE PROCEDURE aplicar_descuento_categoria(IN p_cat VARCHAR(50), IN p_porcentaje  
DECIMAL(5,2))  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN SELECT 'Error al actualizar  
precios' AS mensaje; END;  
  
    UPDATE VEHICULO SET precioPorDia = precioPorDia * (1 + (p_porcentaje/100))  
    WHERE categoria = p_cat;  
    SELECT CONCAT('Precios actualizados para la categoría: ', p_cat) AS mensaje;  
END $$  
DELIMITER ;
```

FUNCIONES:

```
-- FUNCIÓN total_recaudado_categoria  
-- Calcula el dinero total generado por una categoría (JOIN entre 3 tablas).  
DELIMITER $$  
CREATE FUNCTION total_recaudado_categoria(p_cat VARCHAR(50)) RETURNS  
DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    DECLARE v_total DECIMAL(10,2);  
    SELECT SUM(a.precio) INTO v_total  
    FROM ALQUILER a  
    JOIN VEHICULO v ON a.matricula = v.matricula  
    JOIN CATEGORIA c ON v.categoria = c.nombre  
    WHERE c.nombre = p_cat AND a.estadoA = 'Completado';  
    RETURN IFNULL(v_total, 0);  
END $$  
DELIMITER ;
```

```
-- FUNCIÓN nombre_cliente_alquiler
-- Devuelve el nombre del cliente asociado a un ID de alquiler (JOIN).
DELIMITER $$
CREATE FUNCTION nombre_cliente_alquiler(p_idA INT) RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    DECLARE v_nombre VARCHAR(100);
    SELECT c.nombre INTO v_nombre
    FROM CLIENTE c JOIN ALQUILER a ON c.dni = a.dni
    WHERE a.idA = p_idA;
    RETURN IFNULL(v_nombre, 'No encontrado');
END $$
DELIMITER ;

-- FUNCIÓN dias_alquiler_previstos
-- Calcula los días de duración prevista de un alquiler.
DELIMITER $$
CREATE FUNCTION dias_alquiler_previstos(p_idA INT) RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE f_inicio DATE;
    DECLARE f_fin DATE;
    SELECT fechaInicio, fechaDevolucionPrevista INTO f_inicio, f_fin FROM ALQUILER
    WHERE idA = p_idA;
    RETURN DATEDIFF(f_fin, f_inicio);
END $$
DELIMITER ;

-- FUNCIÓN verificar_stock_vehiculo
-- Devuelve un booleano sobre si un vehículo existe y está disponible.
DELIMITER $$
CREATE FUNCTION verificar_stock_vehiculo(p_matricula VARCHAR(15)) RETURNS
BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE v_estado VARCHAR(50);
    SELECT estadoV INTO v_estado FROM VEHICULO WHERE matricula = p_matricula;
    IF v_estado = 'Disponible' THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END $$
DELIMITER ;
```

Aplicación

Para el diseño de la aplicación, se han creado los siguientes diagramas UML:

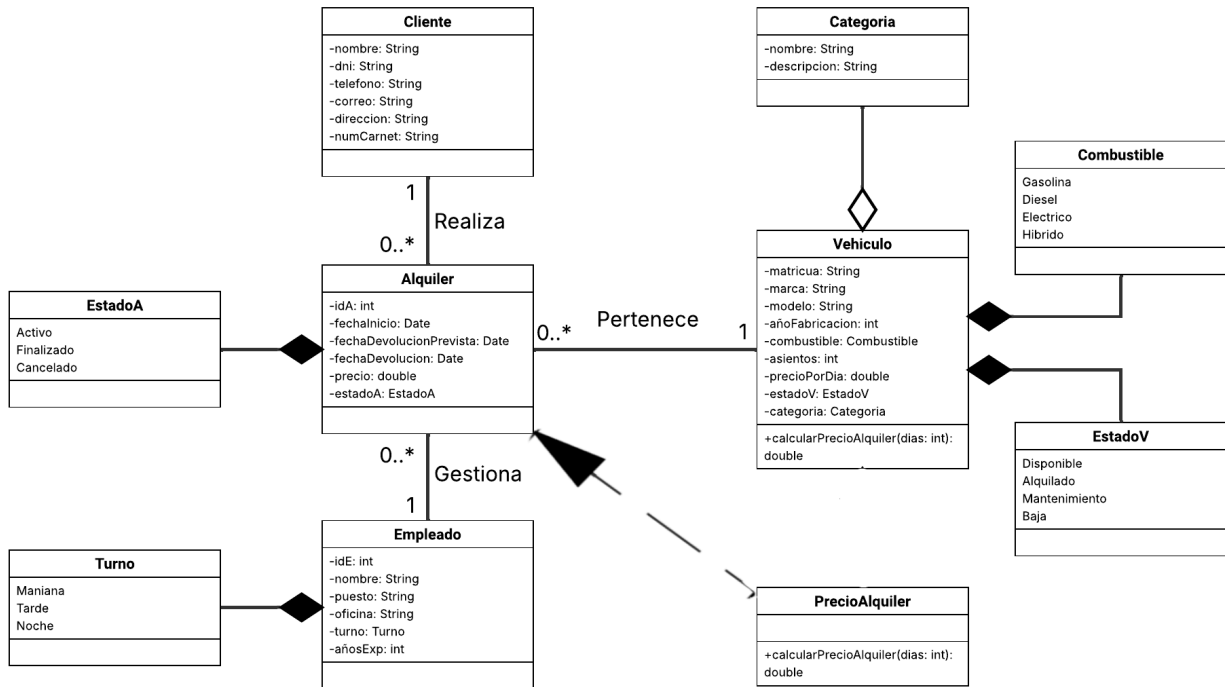


Diagrama de clases

El diagrama de clases ha cambiado respecto al sprint 1 de programación, ahora la interfaz es utilizada por Alquiler.

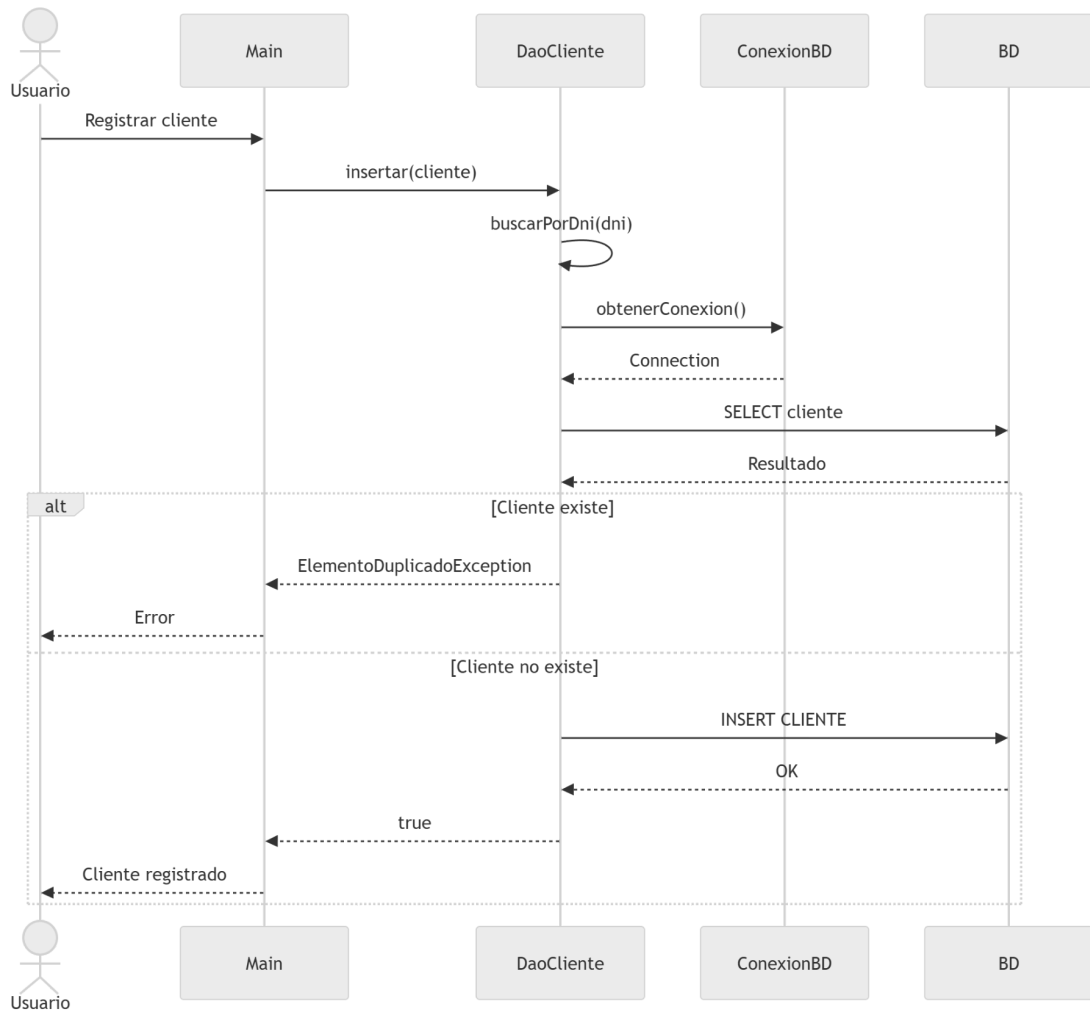
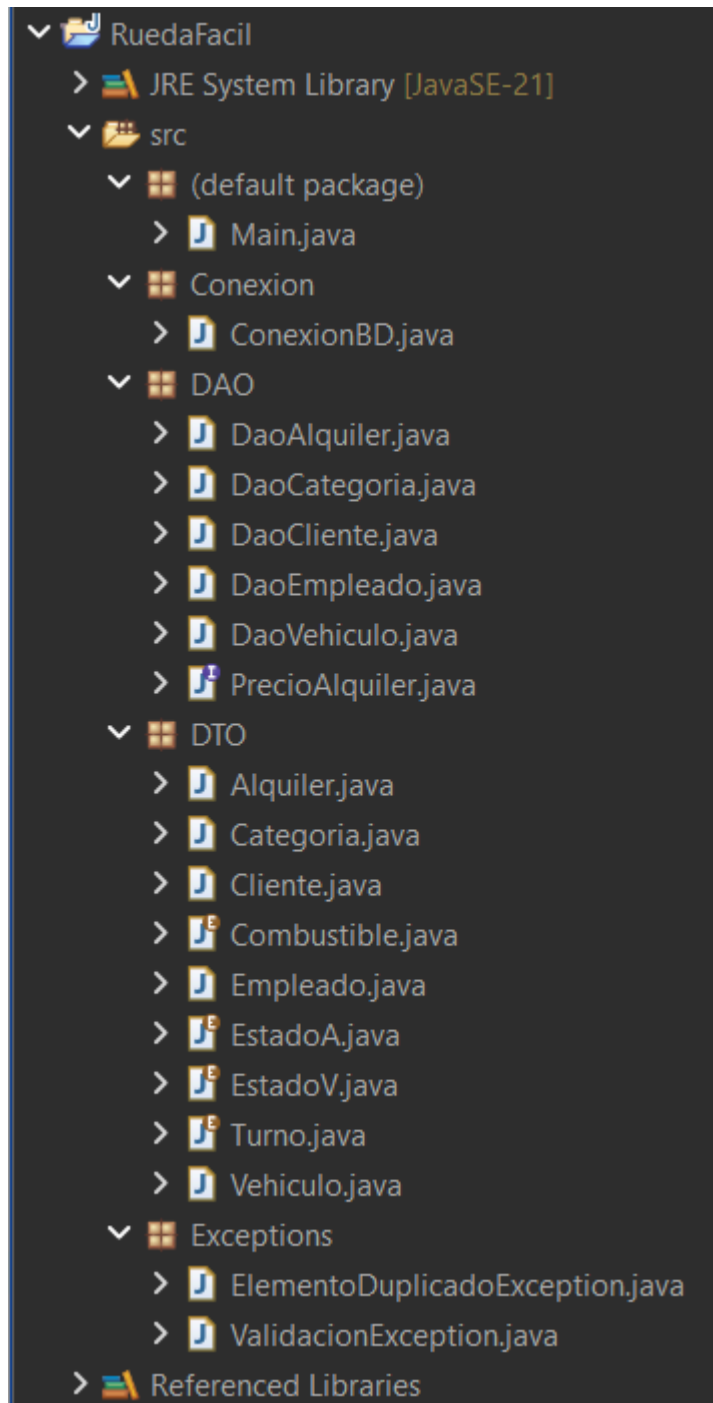


Diagrama de secuencia

El diagrama de secuencia representa de forma visual una función completa del programa.

El proyecto se ha desarrollado siguiendo el patrón de diseño DAO (Data Access Object):



- DAO: Clases de métodos CRUD
- DTO: Clases de objetos, enums e interfaces relacionadas
- default package: Clase principal
- Conexion: Clase de conexión a la base de datos
- Exceptions: Excepciones personalizadas

3. DOCUMENTACIÓN DEL SISTEMA

3.1. MANUAL DE INSTALACIÓN

Antes de ejecutar la aplicación, para su correcto funcionamiento, se deben tener instaladas dos librerías adicionales:

MySQL Connector (para comunicar el programa Java con la base de datos):

<https://downloads.mysql.com/archives/c-j/>

MySQL Product Archives

MySQL Connector/J (Archived Versions)

Please note that these are old versions. New releases will have recent bug fixes and features!
To download the latest release of MySQL Connector/J, please visit [MySQL Downloads](#).

Product Version:
Operating System:

Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-8.4.0.tar.gz)	Mar 12, 2024	4.1M	Download
		MD5: 33a74d1a803b504a1141a327bf6ff7c3 Signature	
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-8.4.0.zip)	Mar 12, 2024	4.9M	Download
		MD5: 5d426b639f7b4c314492edff07c615dc Signature	

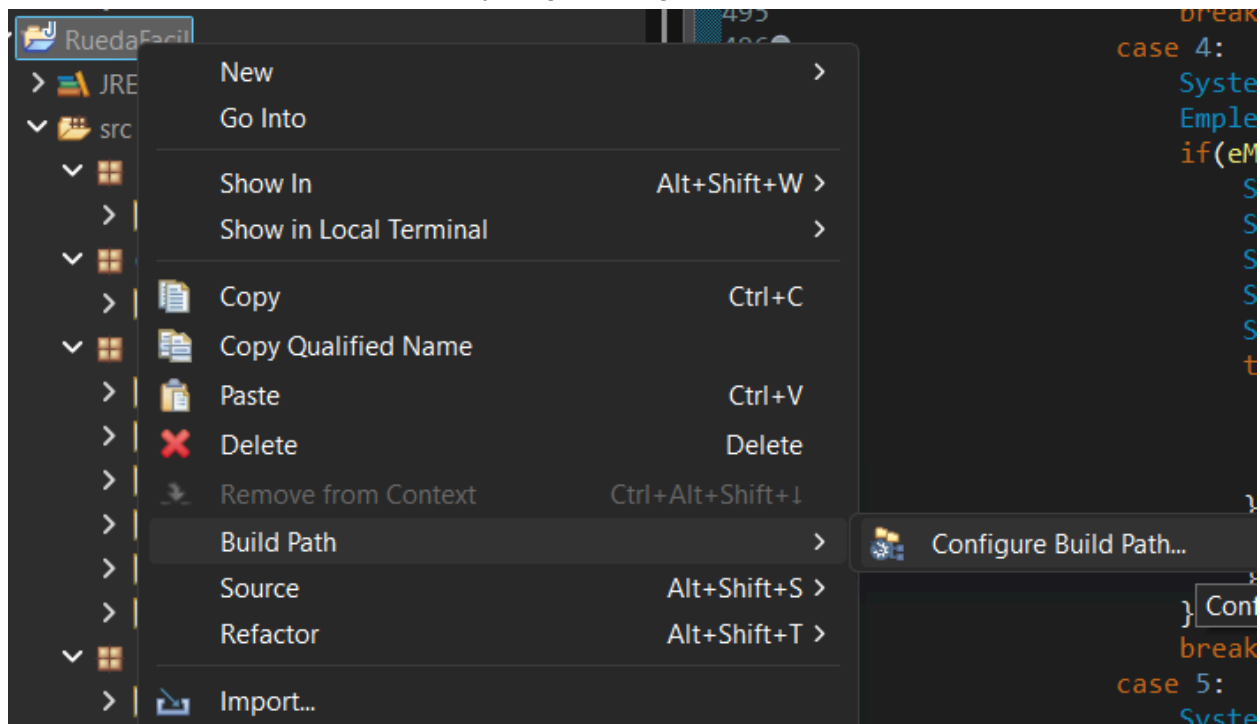
We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

MySQL open source software is provided under the [GPL License](#).

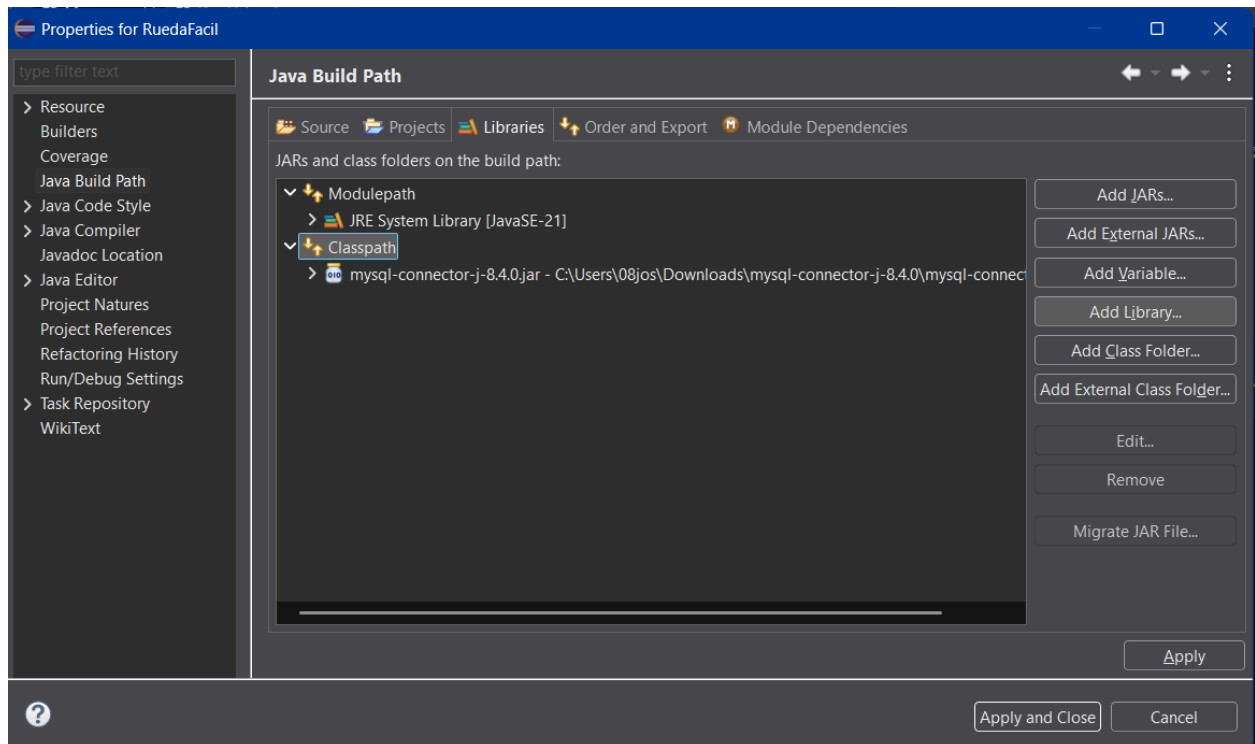
ORACLE © 2026 Oracle

[Privacy](#) / [Do Not Sell My Info](#) | [Terms of Use](#) | [Trademark Policy](#) | [Preferencias sobre cookies](#)

Una vez se descargue, se debe hacer click sobre el proyecto java. Tras eso, se debe seleccionar la opción de “build path” y luego “configure build path”:



Se abrirá un menú. Arriba se debe seleccionar la opción “Libraries” y dentro del recuadro central, se debe seleccionar “Classpath”. A la derecha, se debe seleccionar “Add external JARs”:



Esto abrirá el explorador de archivos, donde se deberá buscar la librería deseada (MySQL Connector):

Nombre	Fecha de modificación	Tipo	Tamaño
al principio de este año			
mysql-connector-j-8.4.0.jar	27/04/2026 10:04	Executable Jar File	2.475 KB
src	27/04/2026 10:05	Carpeta de archivos	

3.2. MANUAL DE USUARIO

MANUAL DE USUARIO: SISTEMA DE GESTIÓN RUEDAFÁCIL

MENU PRINCIPAL

- Opción 1: Accede al submódulo de Gestión de Clientes.
- Opción 2: Accede al submódulo de Gestión de Vehículos.
- Opción 3: Accede al submódulo de Gestión de Alquileres.

- Opción 4: Accede al submódulo de Gestión de Categorías.
- Opción 5: Accede al submódulo de Gestión de Empleados.
- Opción 0: Finaliza la ejecución del programa RuedaFácil.

SUBMENU 1: GESTIÓN DE CLIENTES

- Opción 1 (Registrar cliente): Registra un nuevo cliente tras introducir los datos obligatorios.
- Opción 2 (Buscar cliente por DNI): Muestra la información completa de un cliente tras introducir su DNI.
- Opción 3 (Listar todos los clientes): Muestra una lista con la información de todos los clientes registrados.
- Opción 4 (Modificar cliente): Permite editar la información de un cliente tras introducir su DNI.
- Opción 5 (Eliminar cliente): Elimina un cliente tras introducir el DNI

SUBMENU 2: GESTIÓN DE VEHÍCULOS

- Opción 1 (Registrar vehículo): Registra un nuevo vehículo tras rellenar los datos obligatorios.
- Opción 2 (Buscar vehículos disponibles por categoría): Muestra una lista de vehículos disponibles filtrados tras introducir la categoría específica.
- Opción 3 (Buscar vehículo por matrícula): Muestra la información completa de un vehículo tras introducir su matrícula.
- Opción 4 (Listar flota de vehículos): Muestra una lista de todos los vehículos registrados.
- Opción 5 (Modificar vehículo): Permite editar la información del vehículo tras introducir su matrícula.
- Opción 6 (Eliminar vehículo): Elimina un vehículo del sistema tras introducir la matrícula

SUBMENU 3: GESTIÓN DE ALQUILERES

- Opción 1 (Registrar alquiler): Registra un nuevo alquiler tras rellenar los datos obligatorios.
- Opción 2 (Consultar historial de cliente): Muestra la información de todos los alquileres de un cliente específico tras introducir su DNI.
- Opción 3 (Buscar alquiler por ID): Muestra la información de un alquiler tras introducir su ID.
- Opción 4 (Listar todos los alquileres): Muestra una lista de todos los alquileres registrados.
- Opción 5 (Modificar alquiler - Devolución): Registra el cierre de un alquiler activo asignando, actualizando el estado de la activo a finalizado tras introducir su ID.
- Opción 6 (Eliminar alquiler): Elimina un alquiler tras introducir su ID.

SUBMENU 4: GESTIÓN DE CATEGORÍAS

- Opción 1 (Registrar categoría): Introduce una nueva categoría de vehículo rellenando los datos obligatorios.
- Opción 2 (Buscar categoría por nombre): Muestra la información completa de una categoría concreta introduciendo su nombre.
- Opción 3 (Listar todas las categorías): Despliega en una lista todas las categorías en el sistema.

- Opción 4 (Modificar categoría): Permite localizar una categoría y redefinir su descripción tras introducir su nombre.
- Opción 5 (Eliminar categoría): Elimina una categoría tras introducir su nombre.

SUBMENU 5: GESTIÓN DE EMPLEADOS

- Opción 1 (Registrar empleado): Registra a un nuevo empleado en el sistema rellenando los datos obligatorios.
- Opción 2 (Buscar empleado por ID): Muestra la información completa de un empleado tras introducir su ID.
- Opción 3 (Listar todos los empleados): Muestra una lista completa de todos los empleados en el sistema.
- Opción 4 (Modificar empleado): Permite actualizar la información de un empleado tras introducir su ID.
- Opción 5 (Eliminar empleado): Elimina un empleado tras introducir su ID.

4. CONCLUSIONES FINALES

4.1. GRADO DE CUMPLIMIENTO DE LOS OBJETIVOS FIJADOS

El programa en su estado actual es completamente funcional. Las funciones CRUD solicitadas en el enunciado son totalmente funcionales, conectando a la base de datos para poder gestionar las diferentes entidades. El programa se comunica con la base de datos a través de las clases DAO, implementando también excepciones personalizadas. La base de datos e inserts se realizaron exitosamente. Las funciones y procedimientos solicitados han sido implementados a la base de datos y funcionan correctamente.

4.2 PROPUESTA DE MODIFICACIONES O AMPLIACIONES FUTURAS DEL SISTEMA IMPLEMENTADO

- Implementación de más filtros a la hora de introducir información en la base de datos, permitiendo únicamente patrones específicos de caracteres para apartados como la matrícula o DNI.
- Implementación de comparadores para poder listar las diferentes entidades, por ejemplo, en orden alfabético.
- Sustituir la actual clase Main basada en menús numéricos de consola por una interfaz de usuario moderna.

5. BIBLIOGRAFÍA

- Apuntes y documentación de la asignatura Base de Datos
- Apuntes y documentación de la asignatura Entornos de Desarrollo
- Apuntes y documentación de la asignatura Programación